DBMS Project Report: Regional Transport Office (RTO) Management System V2

1. Project Title

Regional Transport Office (RTO) Management System - Version 2.0

2. Team Members

SRN	Name
PES1UG23CS620	**SURYA P**
PES1UG23CS631	**TALANKI VENKAT TARUN**

3. Abstract

This project implements a comprehensive **Regional Transport Office (RTO) Management System** designed to streamline vehicle registration, licensing, traffic offense management, and ownership transfers. Built with a **Python** backend and **Streamlit** frontend, it interfaces with a **MySQL** database.

The system features a **Role-Based Access Control (RBAC)** mechanism, providing distinct portals for:

- **Citizens (Vehicle Owners):** To view registered vehicles, apply for licenses, transfer vehicle ownership, and view challan history.
- **Admins (RTO Officers):** To manage records (CRUD operations), issue challans, and view system analytics.

Version 2.0 Enhancements:

- Premium UI with Inter font and glassmorphism design
- Interactive charts for visual analytics

- Virtual RC card for citizens
- Enhanced color and fuel type tracking (JSON)
- Complete admin management suite with all CRUD operations

Key database concepts demonstrated include **Normalization (3NF)**, **ACID Transactions** (for ownership transfers), **Triggers** (for business rule enforcement and auditing), **Stored Procedures** (for complex operations), **Functions**, **Views**, and **JSON Data Types**. The system ensures data integrity and provides a user-friendly interface for all RTO operations.

4. Software Stack Used

- **Frontend Framework:** Streamlit (Python) with Custom CSS
- **Backend Logic:** Python 3.x
- **Database:** MySQL 8.0+
- **Database Connector:** `mysql-connector-python`
- **Data Manipulation:** Pandas
- **Data Visualization:** Streamlit Charts

5. Complete Database Details

The database `rto_management` consists of the following normalized tables:

Table Definitions

1. RTO_Offices

Stores details of RTO locations.

```
CREATE TABLE `RTO_Offices` (
  `OfficeID` int(11) NOT NULL AUTO_INCREMENT,
  `OfficeCode` varchar(10) NOT NULL UNIQUE,
  `City` varchar(100) NOT NULL,
  `Address` varchar(255) NOT NULL,
  PRIMARY KEY (`OfficeID`)
) ENGINE=InnoDB;
```

2. Departments

Departments within the RTO (e.g., Licensing, Administration).

```
CREATE TABLE `Departments` (
  `DepartmentID` int(11) NOT NULL AUTO_INCREMENT,
  `DepartmentName` varchar(100) NOT NULL UNIQUE,
  PRIMARY KEY (`DepartmentID`)
) ENGINE=InnoDB;
```

3. Persons

General registry for all citizens.

```
CREATE TABLE `Persons` (
  `PersonID` int(11) NOT NULL AUTO_INCREMENT,
  `FirstName` varchar(50) NOT NULL,
  `LastName` varchar(50) NOT NULL,
  `DateOfBirth` date NOT NULL,
  `Address` text NOT NULL,
  `PhoneNumber` varchar(15) NOT NULL UNIQUE,
  PRIMARY KEY (`PersonID`)
) ENGINE=InnoDB;
```

4. Employees

RTO staff members.

```
CREATE TABLE `Employees` (
  `EmployeeID` int(11) NOT NULL AUTO_INCREMENT,
  `FirstName` varchar(50) NOT NULL,
  `LastName` varchar(50) NOT NULL,
  `JobTitle` varchar(100) NOT NULL,
  `HireDate` date NOT NULL,
  `OfficeID` int(11) NOT NULL,
  `DepartmentID` int(11) NOT NULL,
  PRIMARY KEY (`EmployeeID`),
  FOREIGN KEY (`OfficeID`) REFERENCES `RTO_Offices` (`OfficeID`),
  FOREIGN KEY (`DepartmentID`) REFERENCES `Departments` (`DepartmentID`)
) ENGINE=InnoDB;
```

5. Vehicles

Registered vehicles with JSON support for technical specs (Color, Fuel Type).

```
CREATE TABLE `Vehicles` (
  `VehicleRegNo` varchar(15) NOT NULL,
  `ChassisNo` varchar(30) NOT NULL UNIQUE,
  `EngineNo` varchar(30) NOT NULL UNIQUE,
  `Make` varchar(50) NOT NULL,
  `Model` varchar(50) NOT NULL,
  `ManufactureYear` int(11) NOT NULL,
  `OwnerID` int(11) NOT NULL,
  `IssuingRTO_ID` int(11) NOT NULL,
  `TechnicalSpecs` JSON,
  PRIMARY KEY (`VehicleRegNo`),
  FOREIGN KEY (`OwnerID`) REFERENCES `Persons` (`PersonID`),
  FOREIGN KEY (`IssuingRTO_ID`) REFERENCES `RTO_Offices` (`OfficeID`)
) ENGINE=InnoDB;
```

6. Licenses

```
CREATE TABLE `Licenses` (
  `LicenseNo` varchar(20) NOT NULL,
  `IssueDate` date NOT NULL,
  `ExpiryDate` date NOT NULL,
  `LicenseClasses` varchar(50) NOT NULL,
  `PersonID` int(11) NOT NULL,
  `IssuingRTO_ID` int(11) NOT NULL,
  PRIMARY KEY (`LicenseNo`),
  FOREIGN KEY (`PersonID`) REFERENCES `Persons` (`PersonID`),
  FOREIGN KEY (`IssuingRTO_ID`) REFERENCES `RTO_Offices` (`OfficeID`)
) ENGINE=InnoDB;
```

7. Offenses

```
CREATE TABLE `Offenses` (
  `OffenseCode` varchar(10) NOT NULL,
  `Description` varchar(255) NOT NULL,
  `StandardFine` decimal(10,2) NOT NULL,
  PRIMARY KEY (`OffenseCode`)
) ENGINE=InnoDB;
```

8. Challans

```
CREATE TABLE `Challans` (
  `ChallanID` int(11) NOT NULL AUTO_INCREMENT,
  `VehicleRegNo` varchar(15) NOT NULL,
  `OffenseCode` varchar(10) NOT NULL,
  `OffenseDateTime` datetime NOT NULL,
  `IssuingOfficer` int(11) NOT NULL,
  `FineAmount` decimal(10,2) NOT NULL,
  PRIMARY KEY (`ChallanID`),
  FOREIGN KEY (`VehicleRegNo`) REFERENCES `Vehicles` (`VehicleRegNo`),
  FOREIGN KEY (`OffenseCode`) REFERENCES `Offenses` (`OffenseCode`),
  FOREIGN KEY (`IssuingOfficer`) REFERENCES `Employees` (`EmployeeID`)
) ENGINE=InnoDB;
```

9. Ownership_Transfers

```
CREATE TABLE `Ownership_Transfers` (
  `TransferID` int(11) NOT NULL AUTO_INCREMENT,
  `VehicleRegNo` varchar(15) NOT NULL,
  `PreviousOwnerID` int(11) NOT NULL,
  `NewOwnerID` int(11) NOT NULL,
  `TransferDate` date NOT NULL,
  PRIMARY KEY (`TransferID`),
  FOREIGN KEY (`VehicleRegNo`) REFERENCES `Vehicles` (`VehicleRegNo`),
  FOREIGN KEY (`PreviousOwnerID`) REFERENCES `Persons` (`PersonID`),
  FOREIGN KEY (`NewOwnerID`) REFERENCES `Persons` (`PersonID`)
) ENGINE=InnoDB;
```

10. Audit_Logs

```
CREATE TABLE `Audit_Logs` (
  `LogID` int(11) NOT NULL AUTO_INCREMENT,
  `TableName` varchar(100) NOT NULL,
  `Operation` varchar(50) NOT NULL,
  `RecordID` varchar(50) NOT NULL,
  `ChangeDetails` JSON,
  `LogTime` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (`LogID`)
) ENGINE=InnoDB;
```

6. Advanced MySQL Features Implementation

Views

v_vehicle_details

Aggregates vehicle, owner, and RTO information for efficient retrieval.

```
CREATE VIEW v_vehicle_details AS
SELECT v.VehicleRegNo, v.ChassisNo, v.EngineNo, v.Make, v.Model, v.Manufactur
       CONCAT(p.FirstName, ' ', p.LastName) AS OwnerName,
       p.PersonID AS OwnerID,
       p.PhoneNumber AS OwnerPhone,
       o.City AS RTOCity, o.OfficeCode,
       v.TechnicalSpecs
FROM Vehicles v
JOIN Persons p ON v.OwnerID = p.PersonID
JOIN RTO_Offices o ON v.IssuingRTO_ID = o.OfficeID;
```

Triggers

1. prevent_underage_license

Goal: Prevents license issuance to persons under 18 years old.

```
CREATE TRIGGER prevent_underage_license BEFORE INSERT ON Licenses FOR EACH RO
    DECLARE person_age INT;
    SELECT TIMESTAMPDIFF(YEAR, DateOfBirth, CURDATE()) INTO person_age
    FROM Persons WHERE PersonID = NEW.PersonID;
    IF person_age < 18 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Person must be at least 1
    END IF;
END;
```

2. audit_ownership_transfer

Goal: Automatically logs ownership transfers into `Audit_Logs`.

```
CREATE TRIGGER audit_ownership_transfer AFTER INSERT ON Ownership_Transfers F
    INSERT INTO Audit_Logs (TableName, Operation, RecordID, ChangeDetails)
    VALUES ('Ownership_Transfers', 'INSERT', CAST(NEW.TransferID AS CHAR),
            JSON_OBJECT('vehicle', NEW.VehicleRegNo, 'from', NEW.PreviousOwne
END;
```

3. audit_vehicle_update

Goal: Tracks updates to the `Vehicles` table.

```
CREATE TRIGGER audit_vehicle_update AFTER UPDATE ON Vehicles FOR EACH ROW BEG
    INSERT INTO Audit_Logs (TableName, Operation, RecordID, ChangeDetails)
    VALUES ('Vehicles', 'UPDATE', NEW.VehicleRegNo,
            JSON_OBJECT('old_owner', OLD.OwnerID, 'new_owner', NEW.OwnerID, '
END;
```

Stored Procedures

1. TransferVehicleOwnership

Goal: Atomically transfers ownership. Uses `START TRANSACTION`, `COMMIT`, and `ROLLBACK` to ensure data consistency.

```
CREATE PROCEDURE TransferVehicleOwnership(IN p_RegNo VARCHAR(15), IN p_NewOwr
BEGIN
    DECLARE p_OldOwner INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Transfer failed due to ir
    END;
    START TRANSACTION;
    SELECT OwnerID INTO p_OldOwner FROM Vehicles WHERE VehicleRegNo = p_RegNo
    UPDATE Vehicles SET OwnerID = p_NewOwner WHERE VehicleRegNo = p_RegNo;
    INSERT INTO Ownership_Transfers (VehicleRegNo, PreviousOwnerID, NewOwnerI
    VALUES (p_RegNo, p_OldOwner, p_NewOwner, CURRENT_DATE);
    COMMIT;
END;
```

2. IssueChallan

Goal: Issues a traffic challan for a vehicle.

```
CREATE PROCEDURE IssueChallan(
    IN p_VehicleRegNo VARCHAR(15),
    IN p_OffenseCode VARCHAR(10),
    IN p_OfficerID INT,
    IN p_FineAmount DECIMAL(10,2)
)
BEGIN
    INSERT INTO Challans (VehicleRegNo, OffenseCode, OffenseDateTime, Issuing
    VALUES (p_VehicleRegNo, p_OffenseCode, NOW(), p_OfficerID, p_FineAmount);
END;
```

3. GetDetailedAudit

Goal: Retrieves audit logs with formatted JSON details.

```
CREATE PROCEDURE GetDetailedAudit()
BEGIN
    SELECT LogID, TableName, Operation, RecordID,
           JSON_PRETTY(ChangeDetails) as Details, LogTime
    FROM Audit_Logs ORDER BY LogTime DESC;
END;
```

Functions

1. GetVehicleCount

```
CREATE FUNCTION GetVehicleCount(pid INT) RETURNS INT DETERMINISTIC READS SQL
BEGIN
    DECLARE c INT;
    SELECT COUNT(*) INTO c FROM Vehicles WHERE OwnerID = pid;
    RETURN c;
END;
```

2. GetChallanCount

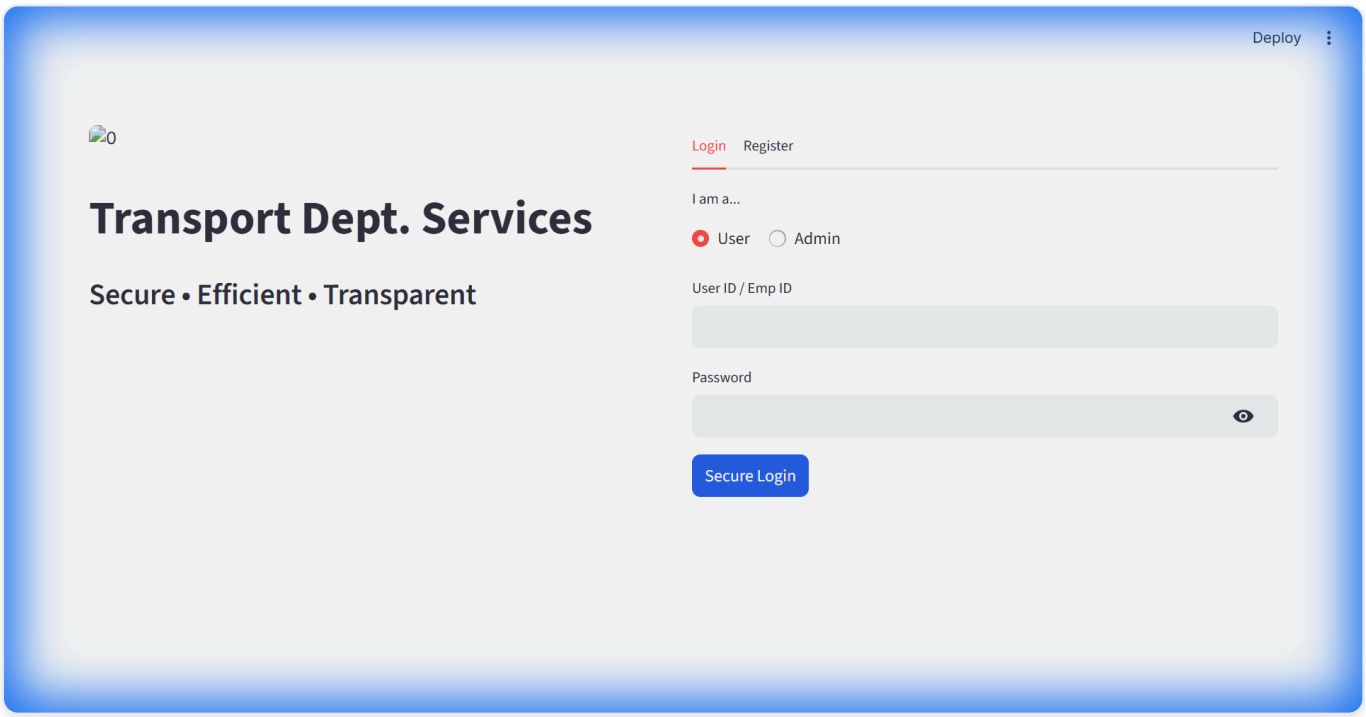
```
CREATE FUNCTION GetChallanCount(pid INT) RETURNS INT DETERMINISTIC READS SQL
BEGIN
    DECLARE c INT;
    SELECT COUNT(*) INTO c FROM Challans WHERE VehicleRegNo IN (SELECT Vehicl
    RETURN c;
END;
```

---

7. Version 2.0 - Feature Documentation

User Portal Features

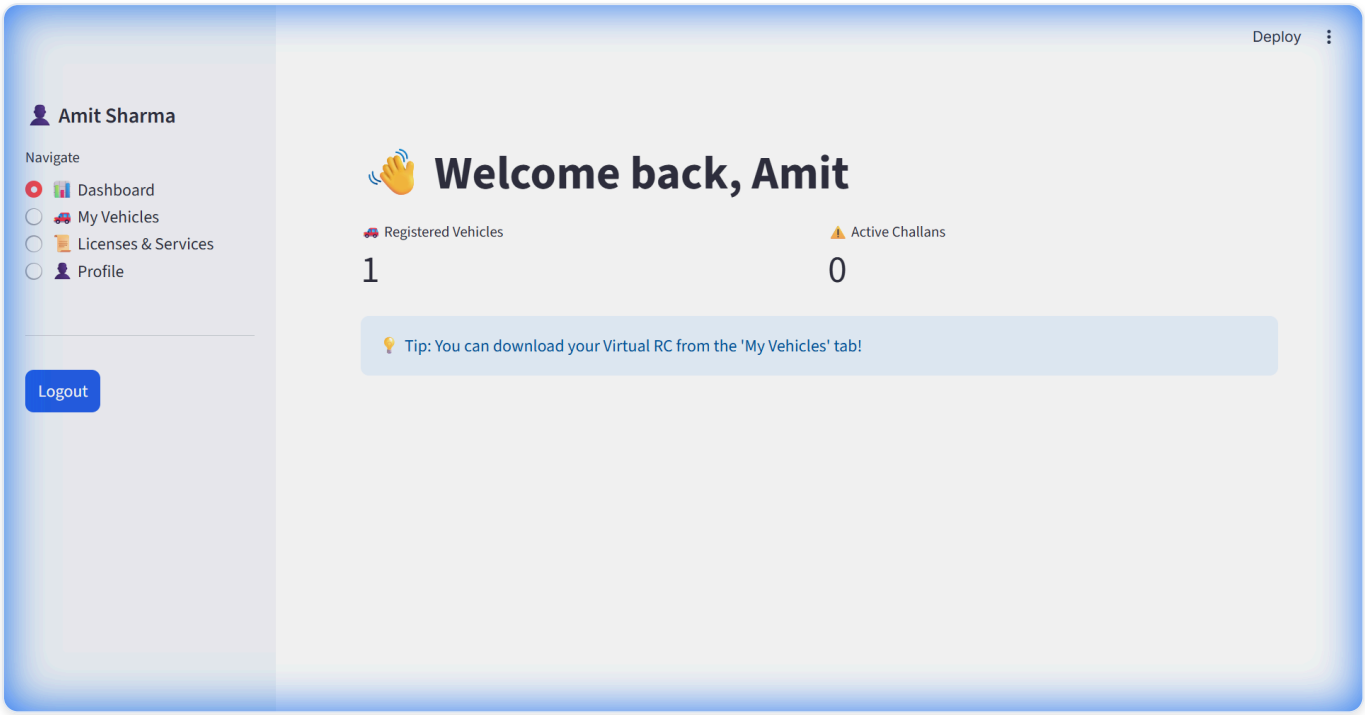
Login Page



Features:

- Dual role selection (User/Admin)
- Secure login with credential validation
- Modern premium UI with glassmorphism design

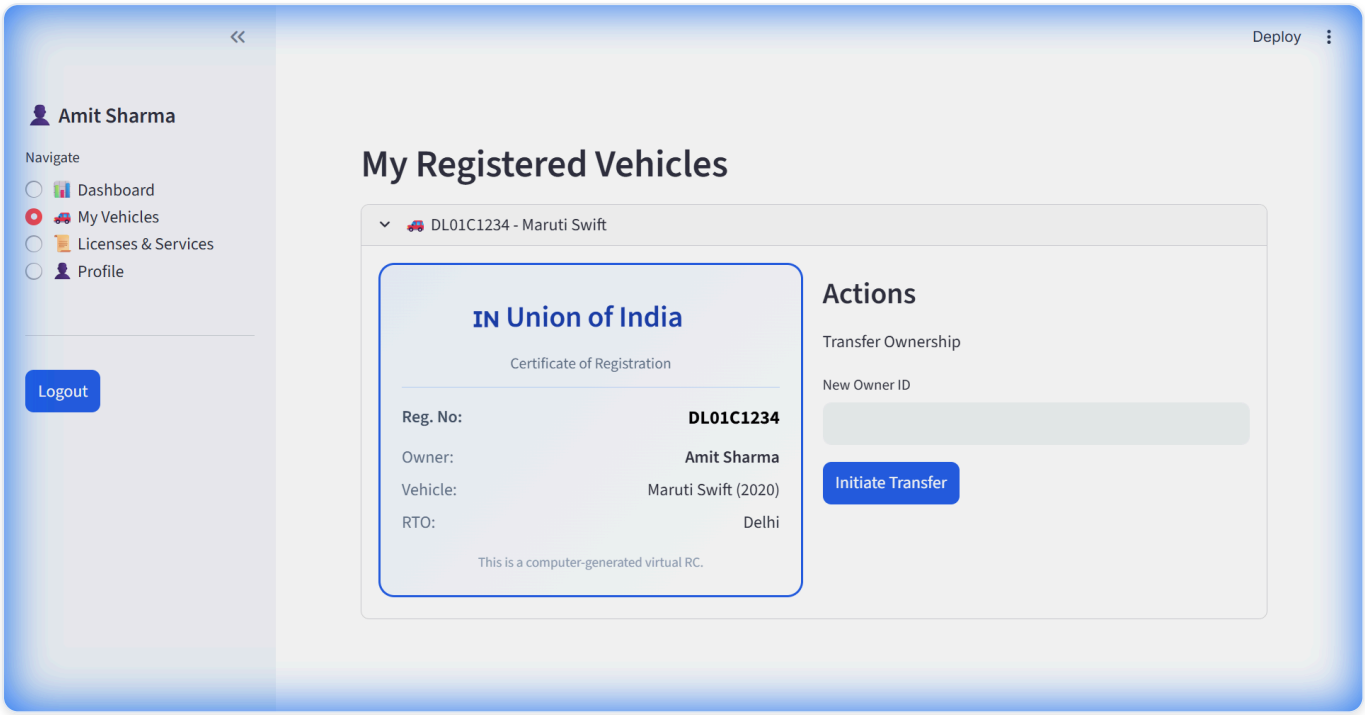
User Dashboard



Features:

- Personalized welcome message
- Metric cards showing registered vehicles and active challans
- Quick tips for users

My Vehicles



Features:

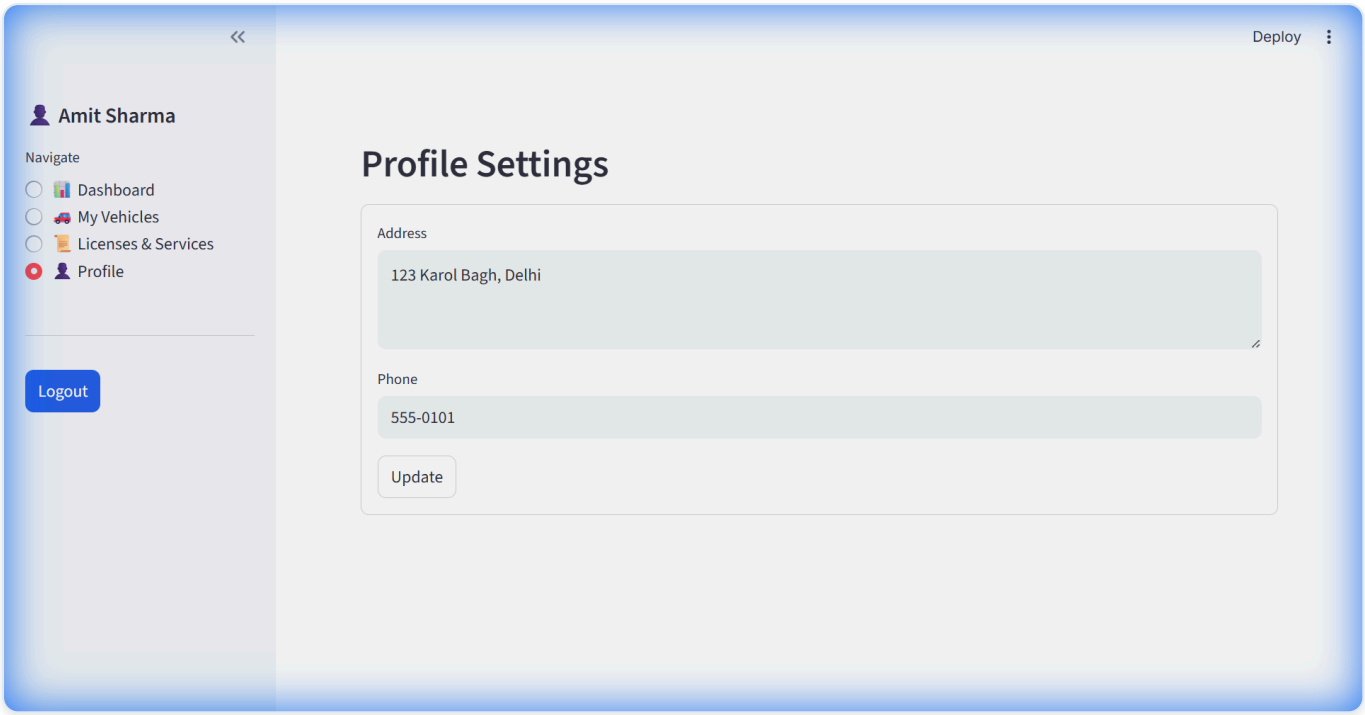
- Virtual RC card with vehicle details
- Transfer ownership functionality
- Clean, card-based design

Licenses & Services

Features:

- License status display
- Challan history tracking
- Apply for new license functionality

User Profile



Features:

- Update address and phone number
- Simple form-based interface

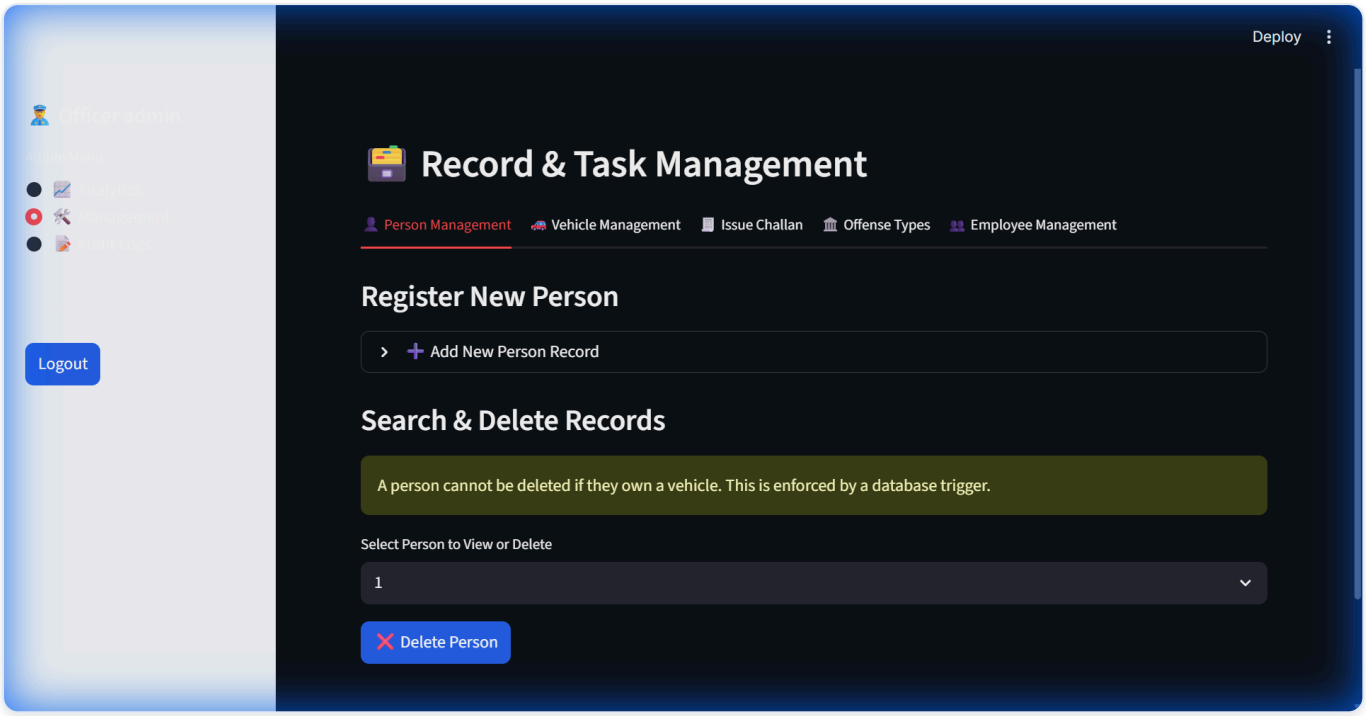
Admin Portal Features

Analytics Dashboard

Features:

- **Key Metrics**: Total Vehicles, Revenue Collected, Registered Citizens
- **Visual Analytics**: Interactive bar chart showing offense distribution
- Real-time data visualization

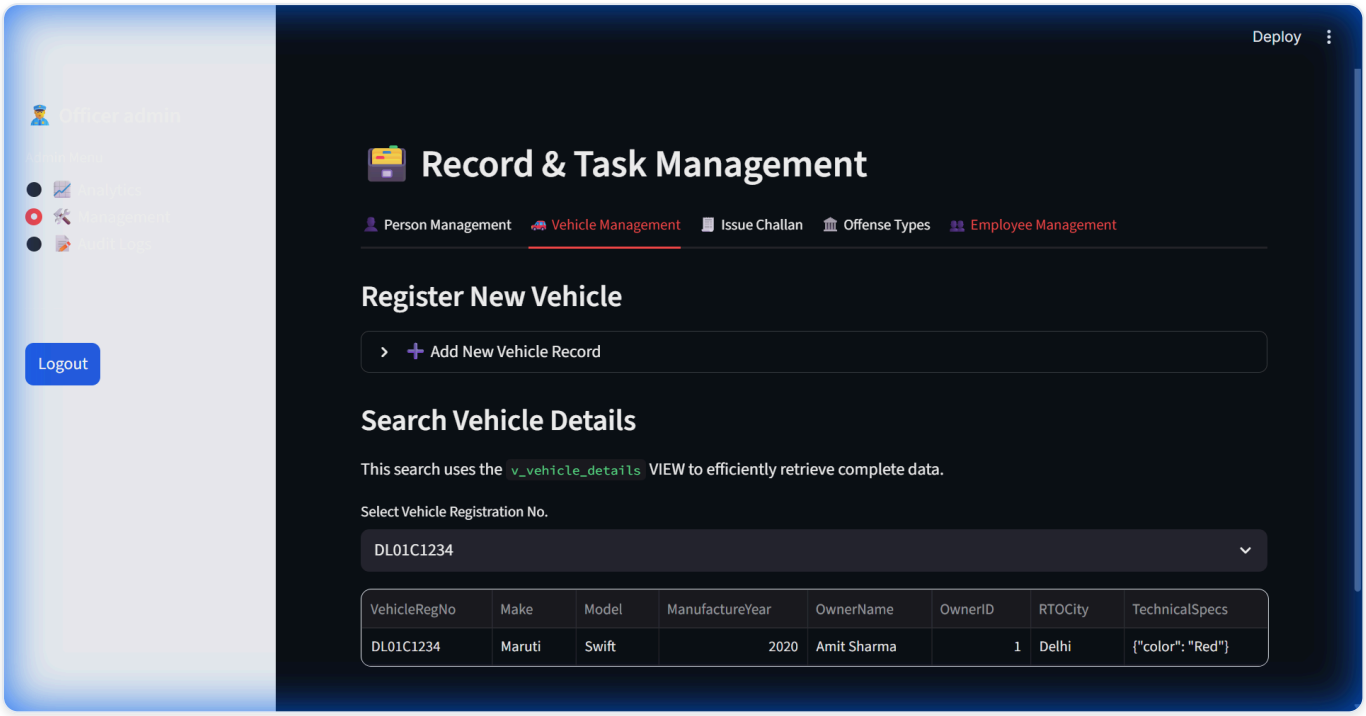
Person Management



Features:

- Register new citizens
- Search and delete person records
- Database trigger protection (cannot delete if owns vehicle)

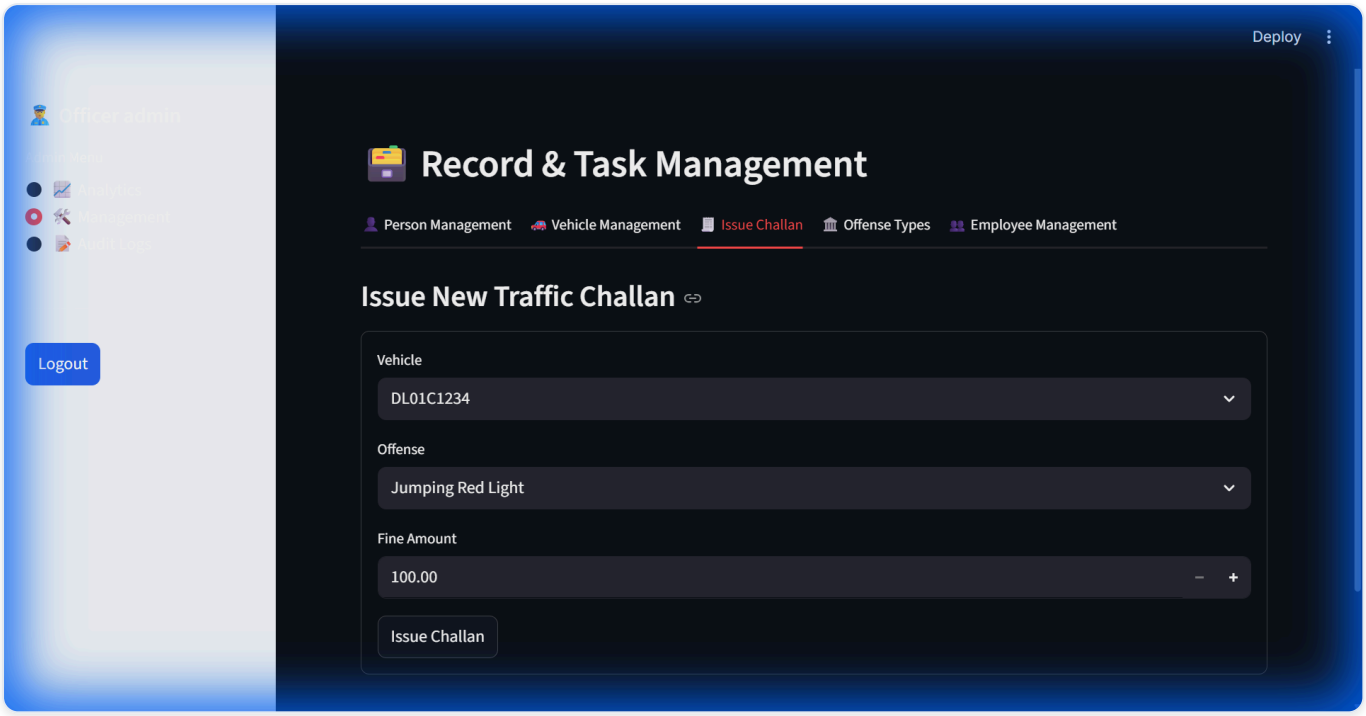
Vehicle Management



Features:

- **Register New Vehicle**: Complete registration form with all required fields
- **V2 Enhancement**: Color and Fuel Type selectors (stored as JSON)
- **Search Vehicle**: Uses `v_vehicle_details` VIEW for efficient data retrieval
- Owner and RTO assignment

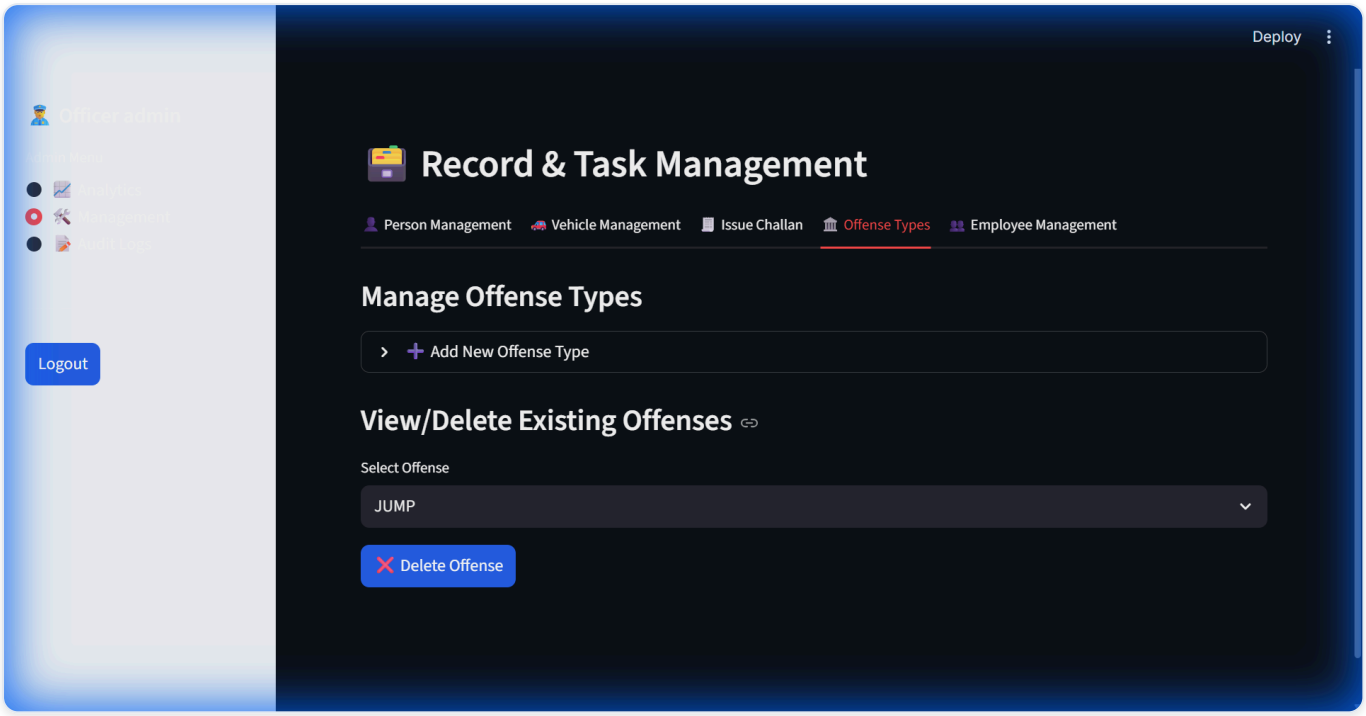
Issue Challan



Features:

- Select vehicle from dropdown
- Choose offense type
- Auto-populated fine amounts
- Uses `IssueChallan` stored procedure

Offense Management



Features:

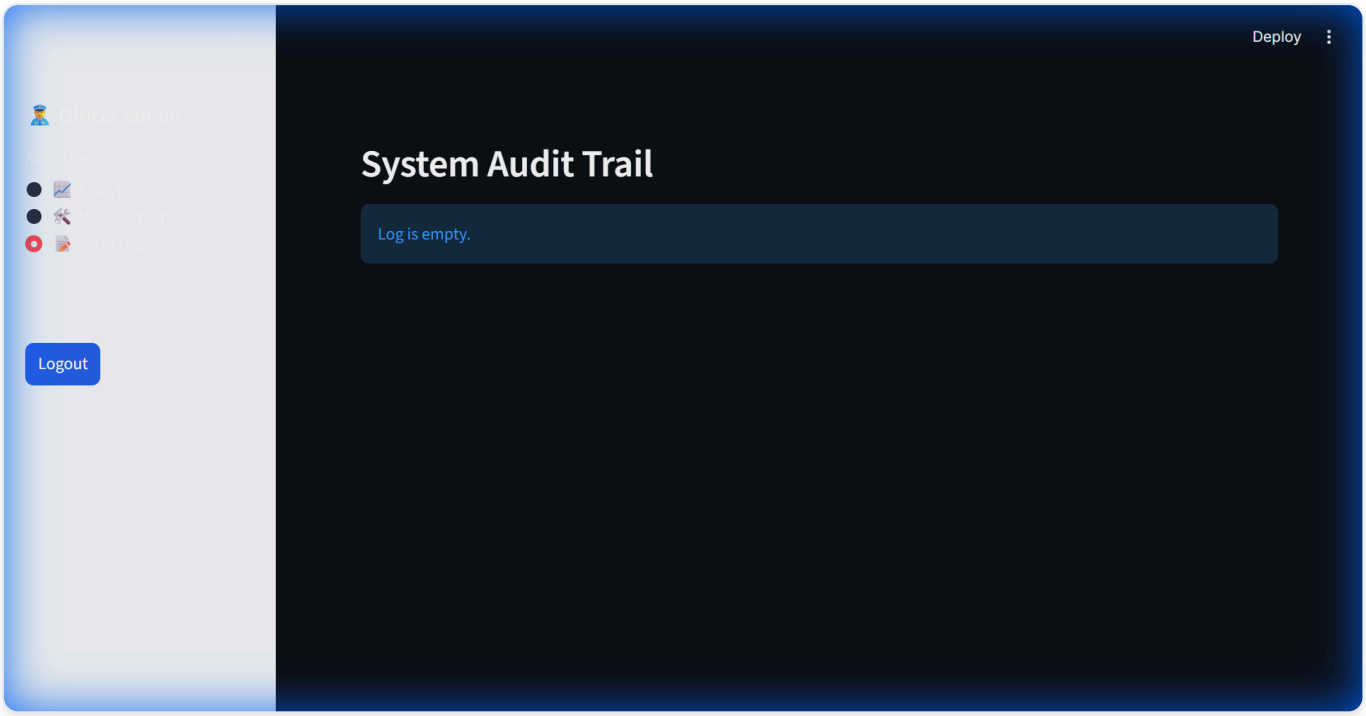
- Add new offense types
- Set standard fine amounts
- Delete existing offenses
- Full CRUD operations

Employee Management

Features:

- Register new employees
- Assign office and department
- View complete employee roster
- Comprehensive employee data display

Audit Logs



Features:

- System audit trail
- Track all database operations
- Searchable audit records
- Timestamp and operation details

---

8. Technical Implementation Summary

Authentication & Authorization

- **Role-Based Access Control (RBAC)**: Separate portals for Users and Admins
- **User Login**: ID-based authentication with phone number verification
- **Admin Login**: ID-based authentication with last name verification (case-insensitive)

V2 Enhancements

- **Premium UI**: Custom CSS with Inter font family and glassmorphism effects
- **Visual Analytics**: Interactive bar charts for offense distribution

- **Virtual RC**: HTML-styled digital registration certificate
- **JSON Data Types**: Vehicle technical specifications (color, fuel type)
- **Enhanced Forms**: Improved UX for all CRUD operations

Database Features Demonstrated

1. **Normalization**: All tables in 3NF
2. **ACID Transactions**: Atomic ownership transfers
3. **Triggers**: Business rule enforcement and audit logging
4. **Stored Procedures**: Complex transactional operations
5. **Functions**: Reusable database logic
6. **Views**: Simplified data retrieval
7. **JSON Support**: Flexible technical specifications

---

9. Login Credentials

User Portal:

- ID: `1`
- Password: `555-0101`

Admin Portal:

- ID: `13`
- Password: `admin`

---

10. Project Files

- `app.py` - Main Streamlit application (V2)
- `mysql_setup.py` - Database setup and initialization
- `db_config.py` - Database connection configuration
- `screenshots/` - All V2 feature screenshots (12 images)
- `Feature_Documentation.md` - Complete V2 feature documentation

- `Project_Report.md` - This comprehensive project report

---

11. Conclusion

The RTO Management System V2 successfully demonstrates advanced database management concepts while providing a modern, user-friendly interface. The system ensures data integrity through triggers and transactions, offers powerful analytics through stored procedures and functions, and delivers an exceptional user experience through the premium V2 UI enhancements.

The project showcases the effective integration of MySQL's advanced features including JSON data types, stored procedures, triggers, and views with a modern Python-based web application framework. The result is a production-ready system that meets both functional requirements and provides an outstanding user experience.